

AlphaExchange

Trading Infrastructure Evaluation Platform

Architecture and Design Document

Repository AlphaExchange

This document covers system architecture, service responsibilities, data flow, security model, scoring algorithm, infrastructure, and all architecture decision records.
Read alongside `DESIGN.md` and `docs/AUDIT.md` in the repository.

Contents

1	Introduction and Platform Overview	5
1.1	Design Imperatives	5
1.2	Repository Layout	5
2	High-Level Architecture	6
2.1	Supporting Infrastructure	7
2.2	Communication Style	7
3	End-to-End Data Flow	7
3.1	Stage 1: Upload	7
3.2	Stage 2: Build	8
3.3	Stage 3: Test Start	8
3.4	Stage 4: Load Generation	9
3.5	Stage 5: Ingestion and Validation	9
3.6	Stage 6: Scoring and Fan-out	9
3.7	Stage 7: Test Completion	10
4	The Sandbox Engine	10
4.1	Layer 1: Network Isolation (Primary Control)	10
4.2	Layer 2: seccomp Profile	10
4.3	Layer 3: AppArmor Profile	11
4.4	Layer 4: Capabilities and Privilege Reduction	11
4.5	Layer 5: Resource Caps	11
4.6	Layer 6: Image Scanning	12
4.7	Layer 7: Runtime Monitoring	12
4.8	Compose vs. Production	12
5	Latency Measurement	12
6	Bot Fleet	13
6.1	Overview	13
6.2	Bot Personas	13
6.3	Shared HTTP Transport	13
6.4	Sequence Number Assignment	14
6.5	Per-Target Circuit Breaker	14
6.6	Two-Phase Test Structure	14
7	Telemetry Ingestion and Validation	15
7.1	At-Least-Once Delivery	15
7.2	Reorder Buffer	15
7.3	Authoritative Reference Order Book	15
7.4	Correctness Validation: Two Modes	16
7.4.1	Mode A: Exact Comparison (Serialized Correctness Phase)	16
7.4.2	Mode B: Hard-Violation Checks (All Phases)	16
7.5	HDR Histogram for Percentile Estimation	17
7.6	TPS Counter	18
7.7	Aggregator and Correctness Rate	18

7.8	TimescaleDB Persistence via COPY	18
8	Real-Time Leaderboard	18
8.1	Composite Scoring Algorithm	18
8.2	Horizontal Fan-Out via Redis Pub/Sub	19
8.3	WebSocket Hub	19
8.4	Freeze and Admin Controls	20
9	Correctness and Anti-Cheat	20
9.1	The Core Problem	20
9.2	Mechanism 1: Serialized Correctness Phase	20
9.3	Mechanism 2: Hard-Violation Checks	21
9.4	Why Both Mechanisms Together	21
10	Orchestrator State Machine	21
10.1	State Transitions	21
10.2	Heartbeat and Orphan Detection	22
11	Inter-Service Communication	22
11.1	Kafka Topics	22
11.2	Protobuf Message Contracts	23
11.3	Redis Data Structures	23
12	Data Stores	24
12.1	PostgreSQL	24
12.2	TimescaleDB	25
12.3	MinIO / S3	25
13	API Contract: Order Book Interface	25
13.1	Endpoints	25
13.2	Semantics	26
14	Observability	26
14.1	Prometheus Metrics	26
14.2	Structured Logging	27
14.3	Distributed Tracing	27
14.4	Grafana Dashboards	27
15	Additional Features	27
15.1	Anomaly Detection	27
15.2	Score Prediction	27
15.3	Contestant Insights	28
15.4	Commentary Ticker	28
15.5	Webhooks	28
15.6	Analysis API	28
16	Infrastructure as Code	28
16.1	Docker Compose (Development)	28
16.2	Helm Charts	29
16.3	Kubernetes Manifests	29

16.4 Terraform	29
17 CI/CD Pipeline	30
17.1 Continuous Integration (<code>ci.yml</code>)	30
17.2 Integration (<code>integration.yml</code>)	30
17.3 Security (<code>security.yml</code>)	31
18 Chaos Engineering	31
19 Architecture Decision Records	31
20 Technology Decisions	33
21 Performance Characteristics	34
22 Security Model Summary	34
22.1 Primary Adversary: Contestant	34
22.2 Secondary Adversary: External Attacker	35
23 Contest Day Runbook	35
23.1 24 Hours Before	35
23.2 One Hour Before	35
23.3 During the Contest	36
23.4 Incident Response	36
23.5 Close	36
24 Correctness Verification	36
24.1 Key Test Coverage Areas	37
25 Design Principles	37
A Environment Variable Reference	38
B Make Targets Reference	39
C Scorer Normalization Example	40

1 Introduction and Platform Overview

AlphaExchange is a distributed benchmarking and hosting platform that evaluates contestant-submitted trading infrastructure under realistic, high-velocity market conditions. Contestants upload an order book implementation; the platform compiles it into a hardened sandbox container, drives it with a fleet of simulated trading bots, validates every fill against an authoritative reference matching engine, and ranks all contestants on a live leaderboard scored across three dimensions: throughput, tail latency, and correctness.

The platform runs as a single-host Docker Compose stack for development and as a horizontally scaled Kubernetes deployment for live contest events. All infrastructure is defined as code and reproducible from a clean checkout.

1.1 Design Imperatives

Three hard requirements shape every component and every architecture decision in this system:

1. **Results must be trustworthy.** A contestant controls the code running inside the sandbox and is motivated to cheat. The platform never trusts the contestant's self-reported fills. Correctness is decided by a reference matching engine that the platform owns and operates, not by the submission.
2. **The platform must survive its own load.** The bot fleet deliberately generates peak market volatility. The measurement and scoring path cannot fall behind, run out of memory, or lose data when a single pod restarts. Backpressure and bounded memory are first-class concerns throughout.
3. **A compromised submission must stay contained.** A contestant who escapes the sandbox must not be able to reach Kafka, Redis, the databases, or other contestants. Isolation is layered so that no single bypassed control constitutes a full breach.

1.2 Repository Layout

```
.
+-- services/                # Go microservices (one module each)
|   +-- submission-api/      # REST ingress and leaderboard read path
|   +-- build-worker/        # Compile and sandbox lifecycle
|   +-- orchestrator/        # Test lifecycle state machine
|   +-- bot-fleet/           # Load generator (all personas)
|   +-- telemetry-ingester/   # Validation, percentiles, persistence
|   +-- leaderboard-api/     # Scoring and WebSocket fan-out
+-- frontend/                # React + Vite + TypeScript SPA
+-- proto/                   # Protobuf message contracts
+-- cmd/generate-template/    # Contestant starter-code generator
+-- testdata/sample-orderbook/ # Reference C++ order book
+-- tests/                   # Cross-service correctness and integration
    tests
+-- scripts/                 # Local dev, migration, smoke, chaos
+-- infra/
|   +-- docker/              # Sandbox seccomp and AppArmor profiles
|   +-- helm/                # Helm charts per service
```

```

|   +-- k8s/                # Raw manifests , network policies , monitoring
|   +-- kafka/              # Topic creation and tuning
|   +-- grafana/            # Dashboards
|   +-- terraform/          # Cloud provisioning
+-- docker-compose.yml      # Full local stack
+-- docker-compose.dev.yml  # Infrastructure-only stack
+-- Makefile                # Developer commands
+-- .env.example             # All environment variables , documented

```

Listing 1: Top-level repository structure

2 High-Level

Architecture

The platform is a polyglot, event-driven set of microservices written in Go, fronted by a React single-page application. Table 1 summarises each service.

Service	Language	Responsibility
submission-api	Go	Authenticated REST ingress. Accepts uploads, streams them to object storage, records submissions, queues build jobs, serves the read path for the leaderboard and contestant-facing data.
build-worker	Go	Consumes build jobs, compiles a submission inside a multi-stage Docker build, launches a hardened sandbox container, waits for its health check, and reports the result.
orchestrator	Go	Owens the test lifecycle state machine. Drives the transition from pending to running to completed or failed, writes heartbeats for crash recovery, triggers final scoring.
bot-fleet	Go	Spawns the load generator. Multiple bot personas send concurrent orders and cancels to the contestant container, measure HTTP round-trip latency, and emit per-order telemetry to Kafka.
telemetry-ingester	Go	Consumes the telemetry firehose, replays orders through the authoritative reference order book to judge correctness, computes latency percentiles and throughput, and persists history and live metrics.
leaderboard-api	Go	Computes the composite score, ranks contestants, and fans the ranked leaderboard out to every connected browser over WebSocket.
frontend	React + TS	Live leaderboard, submission flow, results breakdown, progress and prediction view, and the operations console.

Table 1: Service inventory

2.1 Supporting

Infrastructure

The platform depends on the following infrastructure components, each chosen for a specific role:

- **Apache Kafka / Redpanda** – Durable event backbone for build jobs, telemetry, and test control signals. At-least-once delivery and replay are required for trustworthy scoring; Kafka provides both.
- **Redis** – In-memory store for live per-contestant metrics, the active-contestant set, pub/sub fan-out to leaderboard pods, rate-limit counters, and a read-through submission cache.
- **TimescaleDB** – Time-series hypertable for raw per-order latency samples, throughput snapshots, and per-test summaries. Columnar compression and automatic chunk management handle the highest-volume table efficiently.
- **PostgreSQL** – Strong-consistency relational store for submission state, test state, contestant records, and webhook registrations. Used wherever transactional guarantees matter.
- **MinIO / Amazon S3** – Object storage for contestant submission artifacts. Binaries stream directly to and from object storage without buffering in application memory.

2.2 Communication

Style

Communication is intentionally mixed based on the durability and freshness requirements of each path:

The *durable, high-volume path* runs over Kafka. Build jobs, build results, test control events, and the bot-telemetry firehose all traverse Kafka topics because at-least-once delivery and replay are required for trustworthy scoring. A pod restart on any of these paths must not silently drop data.

The *live read path* runs over Redis pub/sub and WebSocket. Leaderboard updates are published to a Redis channel every scoring interval; every **leaderboard-api** pod subscribes and broadcasts to its connected browsers. Freshness matters more than durability here because a dropped frame is corrected by the next tick.

3 End-to-End

Data

Flow

A submission travels through the system in seven distinct stages, each described in detail below.

3.1 Stage

1:

Upload

A contestant sends a `POST /v1/submissions` multipart request to **submission-api** with their API key in the `X-API-Key` header. The handler:

1. Authenticates the API key via a constant-time SHA-256 hash comparison against the **contestants** table, with a 60-second Redis read-through cache keyed on the hash.

2. Validates the language field, the filename (no path traversal, no null bytes), and the file extension.
3. Enforces upload size bounds: minimum 1 KB, maximum configurable (default 50 MB).
4. Streams the file directly to MinIO using the `minio-go` SDK. The file is never buffered fully in heap memory.
5. Writes a `Submission` row to PostgreSQL with status `pending`.
6. Publishes a `BuildJob` message to the `build-jobs` Kafka topic.
7. Returns HTTP 202 with the `submission_id`.

A per-contestant submission rate limit (10 per hour) is enforced atomically in Redis via a Lua script that increments a counter and sets its TTL in a single round trip, preventing any gap between increment and expiry.

3.2 Stage 2: Build

A **build-worker** consumer fetches the `BuildJob` message. It commits the Kafka offset only after `ProcessBuild` returns, providing at-least-once delivery. The build pipeline proceeds as follows:

1. Download the artifact from MinIO into a temporary directory under `WORK_DIR`.
2. Inspect the archive for path-traversal entries (any entry containing `..` or starting with `/`) and for decompression-bomb patterns (more than 10,000 entries, total uncompressed size above 500 MB, or a compression ratio above 100:1).
3. Extract the archive with a per-entry byte cap enforced as a hard limit on bytes actually decompressed, not on the header-reported uncompressed size.
4. Run a language-specific multi-stage Docker build with a 600-second timeout (accounting for first-run base-image pulls). The final image carries only the compiled artifact and its minimal runtime; build-time toolchains are excluded.
5. Scan the built image with Trivy for HIGH and CRITICAL CVEs. A CRITICAL finding blocks the launch. In strict mode (`SANDBOX_STRICT=true`), a scan failure also blocks the launch.
6. Launch a hardened sandbox container (full details in Section 4).
7. Wait up to 30 seconds for the contestant's `GET /health` endpoint to return HTTP 200.
8. Mark the submission `ready` in PostgreSQL, publish a `ContainerReadyEvent` to the `orchestrator-events` topic, and register the container in the in-memory manager.

3.3 Stage 3: Test Start

When a test is created via `POST /v1/tests`, the submission API validates ownership, confirms the submission status is `ready`, checks that no active test exists for the contestant, inserts a test row with status `pending`, and publishes a `StartTestEvent` to the `orchestrator-events` topic.

The orchestrator consumes this event, acquires a per-contestant Redis lock (scoped to the expected test duration plus 30 seconds), and performs a compare-and-swap status transition from **pending** to **running** in PostgreSQL. If another orchestrator instance wins the CAS, the event is silently ignored as a duplicate. The orchestrator then:

- Clears the contestant's Redis metrics hash so the leaderboard starts fresh.
- Seeds the contestant name into the metrics hash so the leaderboard can display the real name immediately.
- Persists the sandbox IP, port, and start timestamp in Redis.
- Forwards the **StartTestEvent** to the **bot-fleet** via the same topic.
- Arms a stop timer for the configured test duration.
- Begins writing heartbeats every **HEARTBEAT_INTERVAL_SECONDS** seconds.

3.4 Stage 4: Load Generation

The **bot-fleet** consumer receives the **StartTestEvent** and creates a **TestRunner** for the test. The runner:

1. Calls **POST /reset** on the contestant's sandbox so both the contestant's book and the reference shadow book begin from the same empty state.
2. Runs the **serialized correctness phase** to completion (bounded at 30 seconds). A single logical sender issues one scripted order at a time, waiting for the previous acknowledgment before sending the next. This guarantees the true processing order equals the send order, making the reference replay exact.
3. Spawns the **concurrent load phase** bots across all configured personas. Each bot runs independently, sharing a single warmed HTTP transport with the other bots targeting the same sandbox.

Each bot measures the wall-clock HTTP round-trip for every request and emits an **OrderEvent** to the **bot-telemetry** Kafka topic with a monotonic sequence number assigned at send time (before the request write), the phase label, and the actual fill the contestant returned.

3.5 Stage 5: Ingestion and Validation

The **telemetry-ingester** consumes **bot-telemetry** events, buffers them briefly in a reorder window, sorts each flush by sequence number, and replays the per-contestant stream through the authoritative reference order book. Full details are in Section 7.

3.6 Stage 6: Scoring and Fan-out

The **leaderboard-api** scorer reads per-contestant metric snapshots from Redis on a fixed interval. It normalizes throughput and tail latency across the active field with min-max normalization, combines them with the correctness rate into a composite score, ranks the field, and publishes the ranked leaderboard to the **leaderboard:updates** Redis pub/sub channel. Every **leaderboard-api** pod subscribes to that channel and broadcasts to its connected WebSocket clients.

3.7 Stage 7: Test Completion

When the duration elapses, the orchestrator's stop timer fires, the orchestrator publishes a `StopTestEvent`, waits 10 seconds for telemetry to flush, reads the final metric snapshot from Redis, computes the composite score, transitions the test to `completed` in PostgreSQL, writes the final score, and releases the per-contestant Redis lock.

4 The Sandbox Engine

The sandbox is the most security-sensitive component in the platform. The threat model assumes the contestant is hostile: they may try to escape the container, reach internal infrastructure, exhaust shared resources, or cheat the scoring.

Isolation is layered so that no single bypassed control is the only thing standing between a malicious submission and the rest of the platform.

4.1 Layer 1: Network Isolation (Primary Control)

The contestant container is attached only to an isolated Docker network (`contestant-isolated`) that has `internal: true` in Compose, meaning no route to the host or the internet. On Kubernetes, a default-deny `NetworkPolicy` in the `trade-eval` namespace blocks all egress and allows ingress only from `bot-fleet` and `build-worker` pods on port 8080.

The `build-worker` and `bot-fleet` are dual-homed: they are attached to both the platform network (to reach Kafka and the databases) and the contestant-isolated network (to reach the sandbox by container IP). The contestant container is never placed on the platform network. This is the control that matters most: even if every other layer is bypassed, a compromised container has no network path to Kafka, Redis, PostgreSQL, or MinIO.

4.2 Layer 2: seccomp Profile

Every contestant container runs under a seccomp profile (located at `infra/docker/contestant/seccomp/contestant-profile.json`) whose default action is `SCMP_ACT_ERRNO`. The allow list is sized for a network server: it includes socket operations, file I/O, threading primitives, and clock calls. Dangerous syscalls (including `ptrace`, `mount`, kernel module operations, and roughly 200 others) are denied by default action.

The launcher in `services/build-worker/sandbox.go` reads the seccomp profile content and injects it inline into the Docker `SecurityOpt` slice. If the configured profile path cannot be read, or the file is empty, the launcher returns an error and the build fails. The sandbox never starts unconfined.

```
1 func buildSecurityOpt(cfg Config) ([]string, error) {
2     opts := []string{"no-new-privileges:true"}
3
4     if cfg.SeccompProfile != "" {
5         data, err := os.ReadFile(cfg.SeccompProfile)
6         if err != nil {
```

```

7         return nil, fmt.Errorf("seccomp profile %q unreadable: %w",
8             cfg.SeccompProfile, err)
9     }
10    if len(bytes.TrimSpace(data)) == 0 {
11        return nil, fmt.Errorf("seccomp profile %q is empty",
12            cfg.SeccompProfile)
13    }
14    opts = append(opts, "seccomp="+string(data))
15 } else if cfg.StrictSandbox {
16     return nil, errors.New(
17         "strict sandbox: SECCOMP_PROFILE is required but not set")
18 }
19 // ...
20 }

```

Listing 2: Fail-closed seccomp enforcement in sandbox.go

4.3 Layer 3: AppArmor Profile

An AppArmor mandatory-access-control profile (`infra/docker/contestant/apparmor/contestant-profile`) restricts the container to reads of a small set of system files, network activity on TCP only, writes limited to `/tmp`, and explicitly denies writes to `/proc` and `/sys`, denies reads and writes to host pseudo-fileSYSTEMS, and denies `ptrace`. The profile is applied by name at container launch via the `apparmor=` security option. When `SANDBOX_STRICT=true`, a missing or unconfigured AppArmor profile blocks the launch.

4.4 Layer 4: Capabilities and Privilege Reduction

Every contestant container is configured with:

- **CapDrop:** `ALL` – all Linux capabilities dropped.
- **no-new-privileges:** `true` – the process cannot gain additional privileges via `setuid` or file capabilities.
- **ReadonlyRootfs:** `true` – the root filesystem is mounted read-only.
- **/tmp** mounted as a `tmpfs` with `noexec,nosuid,size=64m` – scratch space is available but cannot execute code.
- User `65534:65534` (the conventional `nobody:nogroup`) – the contestant process never runs as root.

4.5 Layer 5: Resource Caps

- **Memory:** configurable cap (default 512 MB); `MemorySwap` set equal to `Memory` to prevent swap usage.
- **CPU:** quota enforced with `CPUQuota=100000` and pinned to specific cores via `CpusetCpus` to prevent one contestant from starving another.
- **Process count:** PID limit of 50, bounding fork bombs.

4.6 Layer 6: Image Scanning

The `security/image_scanner.go` module wraps Trivy. A CRITICAL-severity CVE finding blocks the launch before the sandbox starts. In strict mode, a scan that cannot be completed (Trivy not installed, or the scan errors) also blocks the launch. In development mode, a missing Trivy installation logs a warning and continues.

4.7 Layer 7: Runtime Monitoring

The `security/resource_monitor.go` module polls the container's Docker stats every 5 seconds after launch. It fires an alert callback when memory exceeds 80% of the cap (soft threshold) or when CPU has been pinned at 100% for 30 consecutive seconds.

4.8 Compose vs. Production

In the development Compose stack, AppArmor is not applied because the development host may not have the profile loaded. seccomp is applied (the profile is mounted at `/seccomp/contestant-profile.json`) and `SANDBOX_STRICT` defaults to `false`. In the Kubernetes deployment, `SANDBOX_STRICT=true`, AppArmor profiles are loaded onto every node, and the seccomp JSON is mounted into the build-worker pod. The Kubernetes network policy is the production boundary.

5 Latency Measurement

Latency is measured as the wall-clock HTTP round trip from the instant a bot writes the first byte of the request to the instant it reads the last byte of the acknowledgment. The bot captures a high-resolution `time.Now().UnixNano()` immediately before the send and immediately after the ack; the difference in microseconds is the recorded latency.

```

1 func (c *OrderBookClient) SubmitOrder(
2     ctx context.Context,
3     order OrderRequest,
4 ) (OrderResponse, time.Duration, error) {
5     body, _ := json.Marshal(order)
6     t1 := time.Now()
7     resp, err := c.post(ctx, "/order", body)
8     latency := time.Since(t1)
9     // ...
10 }
```

Listing 3: Latency measurement in the bot client

This measures exactly what the contest cares about: how fast the contestant's server acknowledges an order under load, including the contestant's own queuing and lock contention.

Measurement runs in the bot, not inside the sandbox, so the contestant cannot influence the timer. The shared, pre-warmed connection pool means the TCP handshake cost is paid once per test run and is not charged to individual order measurements.

The platform does not instrument the contestant's process internals. The order-book API contract is a black box: the contestant exposes HTTP endpoints, and the platform

measures and validates only what crosses that boundary. This keeps the contract language-agnostic and prevents the platform from depending on contestant cooperation for honest measurement.

6 Bot

Fleet

6.1 Overview

The **bot-fleet** is the load generator. It consumes **StartTestEvent** messages from Kafka, creates a **TestRunner** per test, and manages the full lifecycle of each bot population until the test completes or is stopped.

6.2 Bot

Personas

Five persona implementations cover different market participant archetypes:

Market Maker Posts symmetric two-sided limit quotes around a drifting mid price and cancels and re-quotes on a short cycle. This is the primary source of resting liquidity. Controlled by `bots/market_maker.go`.

Aggressive Taker Crosses the spread with marketable orders, consuming the liquidity the makers post. Controlled by `bots/aggressive_taker.go`.

Spammer Sends a high rate of small limit orders and immediate cancels of the same order ID to stress throughput and lock contention. Controlled by `bots/spammer.go`.

Whale Sends occasional very large orders (10,000 units) that sweep multiple price levels, testing multi-level matching and partial-fill handling. Controlled by `bots/whale.go`.

FIX Bot Sends self-contained FIX 4.2 messages with correct body length, checksum, and sequence numbers, exercising a protocol path beyond plain REST. Implemented in `bots/fix_bot.go`.

The default persona distribution across the bot population is: market maker 40%, aggressive taker 30%, spammer 20%, whale 10%.

6.3 Shared

HTTP

Transport

All bots targeting one contestant share a single `*http.Transport` constructed in `bots/http_client.go`. A new client per bot would exhaust file descriptors and add a TCP handshake to every latency measurement. The shared transport is configured with:

- `MaxIdleConns`, `MaxIdleConnsPerHost`, `MaxConnsPerHost` all set to 200.
 - `IdleConnTimeout` of 90 seconds.
 - `DisableCompression`: `true` to avoid compression overhead distorting latency measurements.
 - `ForceAttemptHTTP2`: `false` to use HTTP/1.1.
-

6.4 Sequence Number Assignment

Each bot goroutine calls `nextSeq()` to obtain a monotonic sequence number. This call happens *immediately before the request write*, not after the acknowledgment. Assigning after the ack would produce completion-order sequence numbers rather than send-order sequence numbers, defeating the purpose of the reorder buffer in the telemetry ingester.

```

1 // nextSeq returns the next monotonic sequence number. Callers must
  // take the
2 // sequence at send time, immediately before writing the request, so
  // the number
3 // reflects the order the platform issued requests rather than the
  // order acks
4 // completed.
5 func (d Deps) nextSeq() int64 { return d.Seq.Add(1) }
```

Listing 4: Send-time sequencing in `base_bot.go`

6.5 Per-Target Circuit Breaker

A `CircuitBreaker` in `client/circuit_breaker.go` protects both the platform and measurement quality. It operates with three states:

Closed All requests pass through.

Open Requests are rejected fast for `openCooldown` (default 5 seconds) after the failure rate exceeds `failureRate` (default 50%) over a rolling 10-second window.

Half-Open After the cooldown, probe requests are sent. Three consecutive successes (`halfOpenNeeded`) close the breaker; any failure reopens it.

A *minimum-traffic guard* (`minTraffic` = 20) prevents the breaker from tripping before the test has produced a meaningful sample. This ensures a sandbox that is slow to warm up does not trip the breaker before the test really starts.

6.6 Two-Phase Test Structure

The `TestRunner` in `test_runner.go` runs two sequential phases:

1. **Serialized Correctness Phase** (`bots/correctness_phase.go`): One logical sender, one request in flight at a time, bounded at 30 seconds. The script exercises every reachable outcome: resting limits (PENDING), crossing fills (FILLED), multi-level partials (PARTIAL), market into empty book (REJECTED), cancel of resting (CANCELLED), and cancel of already-removed (NOT_FOUND). Because each order is sent only after the previous ack, the true processing order equals the send order.
2. **Concurrent Load Phase**: Spawns the full configured bot population across all personas. Bots run concurrently and independently for the remainder of the test duration.

The correctness phase is guaranteed to run to completion before any load bot starts. Every correctness event therefore carries a lower sequence number than every load event, so the ingester's reorder buffer replays the serialized stream first and judges it exactly.

7 Telemetry Ingestion and Validation

This is the heart of trustworthiness. The ingester has two jobs that must not be conflated: measuring performance, and judging correctness.

7.1 At-Least-Once

Delivery

The Kafka consumer uses `FetchMessage` rather than `ReadMessage`. Offsets are committed explicitly by calling `CommitMessages` only after the reorder buffer has flushed the corresponding events into the pipeline. The pipeline path is: event parsed, batch flushed to TimescaleDB via COPY, metrics updated in memory, Redis written. Only after all of that completes are the offsets committed.

This gives at-least-once delivery: a pod restart re-reads the uncommitted tail, redelivering events that were fetched but not yet handled. Duplicate redeliveries are handled at query time by deduplication on the natural key (`sent_at_ns`, `bot_id`, `order_id`).

Auto-commit is explicitly disabled in the Kafka reader configuration:

```

1 r := kafka.NewReader(kafka.ReaderConfig{
2   Brokers:      strings.Split(cfg.KafkaBrokers, ","),
3   GroupID:      cfg.ConsumerGroupID,
4   Topic:        cfg.TelemetryTopic,
5   // CommitInterval is zero (default): disables auto-commit.
6   // Offsets are committed only by Commit().
7   StartOffset:  kafka.FirstOffset,
8 })

```

Listing 5: Consumer configuration with manual commit

7.2 Reorder

Buffer

Telemetry from distributed bots does not arrive at the ingester in any meaningful order. Kafka preserves order only within a partition, and events for one contestant are spread across partitions and bots. The `ReorderBuffer` in `reorder_buffer.go` holds incoming events briefly and flushes batches sorted by sequence number.

The buffer tick is configurable via `REORDER_BUFFER_MS` (default 100 ms). A longer window gives a more complete ordering at the cost of freshness; a shorter window keeps the leaderboard live at the cost of occasionally replaying a late event out of order.

On every flush, the buffer sorts the pending batch by `SequenceNumber`, invokes `flushFn` with the sorted events, and then invokes `commitFn` with all the Kafka messages (including unparseable ones). The order is always flush-then-commit. A crash between the two redelivers the batch on restart rather than dropping it.

7.3 Authoritative

Reference

Order

Book

The reference order book in `shadowbook/order_book.go` is a price-time-priority matching engine that the platform owns and operates. Key implementation choices:

Integer-cent price keys All prices are converted to integer cents (`int64`) for map keys via `priceCents(price float64)`, eliminating float-equality bugs in price-level lookups.

FIFO at each price level The `Orders` slice within each `priceLevel` preserves submission order; the front of the slice is always the earliest resting order.

Ordering-independent fact accumulation The book maintains `minAskEver`, `maxBidEver`, a set of all limit prices ever quoted (`seenPrices`), and a set of all fully-filled order IDs (`filled`). These facts are accumulated across the entire test and are never reset, so the validator can detect impossible fills regardless of the exact interleaving it replays.

The book is reset per contestant per test: when the ingester sees a new `test_id` for a contestant, it creates a fresh `OrderBook`. This mirrors the `POST /reset` the bot fleet issues to the contestant's sandbox at the start of each test.

7.4 Correctness Validation: Two Modes

The `CorrectnessValidator` in `shadowbook/correctness_validator.go` applies two distinct correctness modes:

7.4.1 Mode A: Exact Comparison (Serialized Correctness Phase)

In the `correctness` phase, the true processing order is known (one request at a time, send before ack), so the comparison is exact:

- The contestant's reported `status` must match the reference exactly.
- For `FILLED` or `PARTIAL` fills, the contestant's `filled_quantity` must match within $1e-6$.
- For `FILLED` or `PARTIAL` fills, the contestant's `filled_price` must be within one cent (0.01) of the reference's resting price.
- A missing or empty actual fill is always wrong.

This is the mode the **correctness rate** is computed from. A submission that returns `FILLED` for every order scores near zero because it disagrees with the reference on `PENDING`, `REJECTED`, and `CANCELLED` steps.

7.4.2 Mode B: Hard-Violation Checks (All Phases)

In the `load` phase, the true processing order is not observable because the contestant's server processes concurrent requests. Ordering-sensitive disagreements are tolerated. However, some fills are impossible *regardless of interleaving*, and these always count against a contestant:

1. **Negative quantity:** `actual.Quantity < -1e-6`.
2. **Overfill:** `actual.Quantity > submitted.Quantity + 1e-6` for any non-cancel order.
3. **No liquidity on that side:** a `BUY` fill when no ask ever existed, or a `SELL` fill when no bid ever existed.

4. **Impossible cross:** a LIMIT_BUY fill where the buy price is below the lowest ask ever quoted (`minAskEver`), or a LIMIT_SELL fill where the sell price is above the highest bid ever quoted (`maxBidEver`).
5. **Price never quoted:** a fill at a price that was never quoted as a limit order in the book (`PriceEverQuoted`).

Hard violations count against correctness in every phase. The validator checks them first; if a hard violation is detected, the event is marked `HardViolation: true` regardless of phase.

7.5 HDR Histogram for Percentile Estimation

Latency percentiles are computed from a logarithmic-bucket High Dynamic Range histogram in `latency/hdr_histogram.go`. This is the most important scaling decision in the platform. A naive sort-based approach is $O(n \log n)$ per window and keeps every sample in memory; at a million events per second it eventually runs out of memory.

The HDR histogram:

- Tracks the range $[1 \mu\text{s}, 10 \text{s}]$ at two significant decimal digits of precision ($\approx 1\%$ relative error).
- Records in $O(1)$ by computing the bucket index from the bit length of the value.
- Queries percentiles in a single scan of the counts array.
- Occupies approximately 18 KB of memory per contestant regardless of event volume.
- Supports exact merge of two histograms via `MergeFrom` for the sliding window implementation.

```

1 func (h *HDRHistogram) RecordValue(latencyUs int64) {
2     if latencyUs < 0 {
3         latencyUs = 0
4     }
5     if latencyUs > highestTrackableValue {
6         latencyUs = highestTrackableValue
7     }
8     h.counts[h.countsIndex(latencyUs)]++
9     h.totalCount++
10    h.totalSum += latencyUs
11    // ...
12 }

```

Listing 6: $O(1)$ record in HDRHistogram

A `SlidingWindowHistogram` in `latency/sliding_window.go` wraps 30 per-second HDR histograms in a ring. `Advance()` is called once per second to rotate to the next bucket and reset it. `GetPercentiles()` merges all 30 buckets into a scratch histogram and queries p50, p90, and p99. A cumulative all-time histogram is also maintained so percentile queries work correctly after all sliding-window buckets have expired (for example, reading metrics after a test ends).

7.6 TPS

Counter

The TPS counter in `metrics/tps_counter.go` uses a ring of 60 one-second buckets. Each bucket carries a `sec` field (the Unix epoch second it represents) and a `count` field. On every `Record` call, the bucket at index `sec % 60` is checked: if its `sec` field does not match the current second, the bucket is reset before incrementing. This prevents an inactivity gap longer than 60 seconds from reading stale data as current activity.

`GetCurrentTPS` averages the counts of the 5 most recently completed buckets. `GetPeakTPS` returns the highest single-second count among buckets whose `sec` is within the trailing 60-second window.

7.7 Aggregator

and

Correctness

Rate

The `Aggregator` in `metrics/aggregator.go` combines latency, TPS, and correctness per contestant. The correctness rate is computed exclusively from the serialized correctness phase events and from hard violations in any phase:

$$\text{correctness_rate} = \frac{\text{correct_orders}}{\text{correctness_total}}$$

where `correctness_total` is the count of correctness-phase events plus the count of hard violations from the load phase. Load-phase events without a hard violation do not increment either counter. The rate is reported as 0.0 until at least one correctness event has been processed, preventing a misleading 1.0 during the warmup before the correctness phase begins.

7.8 TimescaleDB

Persistence

via

COPY

Raw per-order samples are written to the `latency_samples` hypertable using `pgx.CopyFrom` (the PostgreSQL COPY protocol) rather than batched INSERT statements. COPY sustains a row rate that batched single-row inserts cannot match.

Because COPY cannot apply an `ON CONFLICT` clause, duplicate events from at-least-once redelivery are not deduplicated at write time. Deduplication is handled at query time using the natural key (`sent_at_ns`, `bot_id`, `order_id`).

The `TimescaleWriter` maintains an internal buffer channel of 50,000 samples and a goroutine that flushes batches when either the batch size threshold is reached or the 1-second ticker fires. On shutdown, `Close()` signals the goroutine, drains the remaining buffer, and blocks until the final flush completes.

8 Real-Time

Leaderboard

8.1 Composite

Scoring

Algorithm

The composite score weights three dimensions:

$$\text{score} = (0.40 \times \text{norm}(\text{TPS}) + 0.40 \times (1 - \text{norm}(\text{p99})) + 0.20 \times \text{correctness_rate}) \times 100 \quad (1)$$

where $\text{norm}(x)$ is min-max normalization across the active field:

$$\text{norm}(x) = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

with the convention that $\text{norm}(x) = 1.0$ when $x_{\min} = x_{\max}$.

p99 is inverted (subtracted from 1) because lower latency scores higher. The weighted sum is scaled to 0 to 100 and rounded to two decimal places.

Tie-breaking: ties break first on correctness rate, then on p99 latency.

Disqualification: a contestant with fewer than 100 valid (non-timed-out) orders is disqualified before ranking. Disqualified entries are listed after all ranked entries with score 0 and no rank assignment.

Score normalization is relative: the score is relative to the current field, not to an absolute target. This is the correct model for a live competitive leaderboard.

8.2 Horizontal Fan-Out via Redis Pub/Sub

The scorer publishes the ranked leaderboard to the Redis channel `leaderboard:updates` on every scoring tick. Every `leaderboard-api` pod subscribes to this channel via the `Subscriber` in `pubsub/redis_subscriber.go` and broadcasts each update to its connected WebSocket clients via the local hub.

The scorer never broadcasts directly to its own hub clients; it publishes to the channel and its own subscription receives the message just like any other pod. This design has two consequences:

- There is no double delivery on the scoring pod.
- Pods are completely stateless with respect to scoring; any number of pods can be added behind a load balancer without coordination.

8.3 WebSocket Hub

The Hub in `hub/websocket_hub.go` manages the set of connected browser clients:

Registration New clients are registered via a channel. The hub checks the total client count against `MAX_WEBSOCKET_CONNECTIONS` and closes the connection if the limit is reached.

Initial snapshot On registration, the hub performs a non-blocking send of the last broadcast payload to the new client so new viewers see state immediately.

Broadcast The broadcast performs a *non-blocking send* to each client's buffered channel. If a client's buffer is full, the hub closes and drops that client rather than blocking the broadcast loop. One slow client cannot stall the broadcast for thousands of others.

Ping-pong keepalive A 30-second ping timer per client detects dead connections. Connections that miss a pong within 60 seconds are reaped.

The frontend keeps a **WebSocketManager** with exponential-backoff reconnection (doubling from 1 second up to 30 seconds). It derives the WebSocket scheme (**ws** or **wss**) from the page protocol so connections work behind TLS without mixed-content blocking. The **connect()** method is idempotent, so React StrictMode double-invocations and remounts do not open duplicate sockets. While the WebSocket is disconnected, the store polls **GET /v1/leaderboard** via REST every 3 seconds, stopping automatically when the socket reconnects.

8.4 Freeze and Admin Controls

The leaderboard can be frozen by posting to **POST /admin/v1/leaderboard/freeze** with the admin API key. The scorer checks for the **leaderboard:frozen** key in Redis on every tick and skips the scoring cycle if it exists. The freeze key has a 24-hour TTL as a safety net. All admin endpoints require constant-time API key comparison via **crypto/subtle.ConstantTimeCompare**.

9 Correctness and Anti-Cheat

9.1 The Core Problem

A contestant controls the code and is motivated to cheat. The simplest cheat is to return **FILLED** for every order, claiming 100% correctness regardless of the book state. Two independent mechanisms close this loophole from different directions.

9.2 Mechanism 1: Serialized Correctness Phase

Before any concurrent load starts, the bot fleet runs a single scripted order stream (one in-flight request at a time) through the contestant. The script exercises every reachable outcome:

Order	Expected Status	Why
LIMIT_SELL 101@10	PENDING	No crossing bid; rests on ask side
LIMIT_SELL 102@10	PENDING	Same; second level
LIMIT_BUY 99@10	PENDING	No crossing ask; rests on bid side
LIMIT_BUY 101@5	FILLED	Crosses the ask at 101, fills 5 @ 101
MARKET_BUY @5	FILLED	Consumes remaining 5 of ask@101 @ 101
MARKET_BUY @15	PARTIAL	10 fill from ask@102, 5 unfilled
MARKET_BUY @5	REJECTED	No asks remaining

Order	Expected Status	Why
CANCEL bid	CANCELLED	Resting bid removed
CANCEL bid again	NOT_FOUND	Already removed
MARKET_SELL @5	REJECTED	No bids remaining
LIMIT_SELL 100@20	PENDING	Rests
LIMIT_BUY 100@30	PARTIAL	Fills 20 @ 100; 10 rests
CANCEL re-mainder	CANCELLED	Cancel resting portion

Table 2: Serialized correctness script outcomes

A submission that returns FILLED for everything disagrees with the reference on PENDING, REJECTED, and CANCELLED steps and scores near zero. It is also caught independently by the hard-violation checks.

9.3 Mechanism 2: Hard-Violation Checks

The hard-violation checks run in every phase and use facts accumulated over the whole test. They detect fills that are impossible regardless of concurrent interleaving. A submission cannot claim a fill at a price that was never quoted, cannot fill more than it submitted, and cannot fill on a side that never had any liquidity. These checks are independent of the exact replay order.

9.4 Why Both Mechanisms Together

A submission that detects it is being evaluated in the correctness phase (for example, by observing the serialized request pattern) and behaves correctly only during that phase is caught by the hard-violation checks in the load phase. Conversely, a submission that simply fakes fills regardless of phase is caught by the exact serialized comparison. The two mechanisms close the loophole from orthogonal directions.

10 Orchestrator State Machine

10.1 State Transitions

The test lifecycle is managed by the `TestStateMachine` in `state_machine.go`. Valid transitions are:

`pending → running → stopping → completed`

`pending → failed, running → failed, stopping → failed`

Transitions are performed as optimistic compare-and-swap operations in PostgreSQL:

```
UPDATE tests
SET status = $3,
    updated_at = NOW(),
    orchestrator_instance_id = $4
WHERE id = $1 AND status = $2
```

Listing 7: Optimistic CAS transition in db.go

If zero rows are affected, another instance has already made the transition, and the `ErrConcurrentModification` sentinel is returned. This prevents duplicate test starts when the same event is delivered to multiple orchestrator instances (which can happen with at-least-once Kafka delivery).

10.2 Heartbeat and Orphan Detection

Each orchestrator instance writes a `last_heartbeat_at` timestamp to the test row every `HEARTBEAT_INTERVAL_SECONDS` seconds via `Recovery.WriteHeartbeats`. A separate `Recovery.DetectOrphans` goroutine queries for tests in `running` or `stopping` states whose heartbeat is older than `ORPHAN_DETECTION_SECONDS`:

```
SELECT id, submission_id, contestant_id, status, started_at,
    last_heartbeat_at, orchestrator_instance_id
FROM tests
WHERE status IN ('running', 'stopping')
    AND (last_heartbeat_at IS NULL
        OR last_heartbeat_at < NOW() - make_interval(secs => $1))
```

Listing 8: Orphan detection query

For each orphaned test, the recovering instance:

1. Acquires a Redis recovery lock (`lock:recover:{testID}`) via `SetNX` to ensure only one instance recovers each test.
2. Probes the sandbox health endpoint.
3. If the sandbox is healthy and the test duration has not elapsed, re-registers a stop timer and claims ownership.
4. If the sandbox is unavailable or the duration has elapsed, marks the test failed or stops it cleanly.

A pod restart therefore causes a blip of approximately `ORPHAN_DETECTION_SECONDS` (default 60 seconds) before recovery, not a permanently stuck test.

11 Inter-Service Communication

11.1 Kafka

Topics

Topic	Partitions	Purpose
build-jobs	4	BuildJob messages from submission-api to build-worker
orchestrator-events	4	All test lifecycle events; includes ContainerReady, StartTest, StopTest, ContainerCrashed
bot-telemetry	16	High-volume OrderEvent firehose from bot-fleet to telemetry-ingester; partitioned by contestant ID
leaderboard-updates	2	Unused as a consumed topic; Redis pub/sub is the fan-out path
submissions	4	New submission notifications
telemetry-anomalies	2	Anomaly events from the ingester

Table 3: Kafka topic inventory

All topics use a replication factor of 3 in production, **snappy** compression for the high-volume telemetry topic, and a 1-week retention for build topics.

11.2 Protobuf Message Contracts

Message schemas are defined in `proto/` and shared across services. The key messages are:

BuildJob Queued by submission-api to build-worker: `submission_id`, `s3_key`, `language`.

BuildResult Published by build-worker: `submission_id`, `status`, `error_log`, `container_ip`, `container_port`.

StartTest Orchestrator to bot-fleet: `test_id`, `contestant_id`, `target_ip`, `target_port`, `duration_seconds`, `bot_count`, `bot_personas`.

StopTest Orchestrator to bot-fleet: `test_id`, `reason`.

TestHeartbeat Orchestrator liveness signal: `test_id`, `timestamp_ns`, `active_bots`.

OrderEvent Per-order telemetry on bot-telemetry: full identity fields, timing fields, order details, expected fill, actual fill, correctness verdict, and `sequence_number`. The `phase` field distinguishes the serialized correctness stream ("**correctness**") from the concurrent load stream ("**load**").

LeaderboardEntry One ranked contestant row with all metric fields.

LeaderboardUpdate Full leaderboard snapshot broadcast to clients.

11.3 Redis Data Structures

Key Pattern	Type	Contents
<code>metrics:{contestant_id}</code>	Hash	p50/p90/p99 latencies, TPS, correctness rate, valid orders, test status, contestant name
<code>leaderboard:active_contestants</code>	Set	Contestant IDs currently on the leaderboard
<code>leaderboard:cached</code>	String	Serialized last leaderboard JSON (500 ms TTL)
<code>leaderboard:frozen</code>	String	Present when leaderboard is frozen (24 h TTL)
<code>leaderboard:updates</code>	PubSub	Ranked leaderboard channel
<code>lock:test:{contestant_id}</code>	String	Per-contestant test exclusion lock
<code>lock:recover:{test_id}</code>	String	Per-test recovery exclusion lock (30 s TTL)
<code>orchestrator_heartbeat:{test_id}</code>	String	Instance ID (30 s TTL)
<code>test:{test_id}</code>	Hash	started_at, target_ip, target_port, contestant_id, status
<code>container:{submission_id}</code>	Hash	ip, port, status
<code>anomalies:{contestant_id}</code>	List	Last 100 anomaly reports (capped)
<code>contestant_cache:{hash}</code>	String	Cached contestant record (60 s TTL)
<code>ratelimit:submission:{id}</code>	String	Submission rate-limit counter (1 h TTL)

Table 4: Redis key inventory

12 Data

Stores

12.1 PostgreSQL

Holds small, consistency-critical state. Two separate instances are used:

Orchestrator DB Contains `contestants`, `submissions`, `tests`, `contests`, `contest_tests`, `contest_results`, and `webhooks` tables. Migrations live in `services/orchestrator/migrations/` and are applied idempotently.

TimescaleDB A TimescaleDB-extended PostgreSQL instance that contains `latency_samples` (hypertable), `tps_samples` (hypertable), and `test_summaries`. Migrations live in `services/telemetry-ingester/migrations/`.

All `updated_at` columns are maintained by a shared trigger function `set_updated_at()` applied to each mutable table.

12.2 TimescaleDB

The `latency_samples` hypertable is the highest-volume table in the platform.
Configuration:

```
-- 1-hour chunk intervals
SELECT set_chunk_time_interval('latency_samples', INTERVAL '1 hour');

-- Columnar compression after 7 days
ALTER TABLE latency_samples SET (
    timescaledb.compress,
    timescaledb.compress_orderby = 'time DESC',
    timescaledb.compress_segmentby = 'contestant_id'
);
SELECT add_compression_policy('latency_samples', INTERVAL '7 days');
```

Listing 9: TimescaleDB configuration for `latency_samples`

Key indexes support three query patterns: time-range per contestant (for latency charts), per-test (for summary queries), and contestant-persona breakdown (for analysis). Because COPY is used for ingestion and cannot apply ON CONFLICT, the unique key for at-query-time deduplication is the tuple (`sent_at_ns`, `bot_id`, `order_id`).

12.3 MinIO

/

S3

Contestant submission artifacts are stored in object storage with the key pattern `submissions/{contestant_id}/{submission_id}/{filename}`. In production, the bucket has a 7-day lifecycle expiration rule. Public access is fully blocked. In Kubernetes, the build-worker accesses the bucket using IAM Roles for Service Accounts, so no long-lived cloud credentials live in pods.

13 API Contract: Order Book Interface

Every contestant implements an HTTP server on port 8080. The platform measures and validates only what crosses this boundary.

13.1 Endpoints

POST /order Submit an order.

- Request: {`order_id`, `type`, `price?`, `quantity`}
- Types: `LIMIT_BUY`, `LIMIT_SELL`, `MARKET_BUY`, `MARKET_SELL`
- Response: {`order_id`, `status`, `filled_price`, `filled_quantity`, `remaining_quantity`}
- Statuses: `FILLED`, `PARTIAL`, `PENDING`, `REJECTED`

POST /cancel Cancel a resting order.

- Request: {`order_id`}
 - Response: {`order_id`, `status`}
-

- Statuses: CANCELLED, NOT_FOUND, ALREADY_FILLED

GET /health Liveness probe. Must return HTTP 200 with {"status":"ok"} within 3 seconds of startup.

POST /reset Clear all order book state. Called by the bot fleet at the start of each test.

GET /orderbook Optional debug endpoint returning the current bids and asks.

13.2 Semantics

- Price-time priority: at each price level, earlier orders fill first (FIFO).
- Limit orders that cross fill immediately; the remainder rests in the book.
- Fill prices for limit orders must match the resting price exactly (within one cent).
- Market orders consume the best available opposite-side liquidity; REJECTED if empty.
- The platform times out after 5 seconds per order; timed-out orders are excluded from the correctness rate denominator.

14 Observability

14.1 Prometheus

Metrics

Each service exposes **GET /metrics** for Prometheus scraping. Key metrics by service:

Service	Metric	Type
submission-api	submission_api_http_requests_total	Counter (method, route, status)
submission-api	submission_api_http_request_duration_seconds	Histogram
orchestrator	orchestrator_tests_started_total	Counter
orchestrator	orchestrator_tests_completed_total	Counter
orchestrator	orchestrator_tests_failed_total	Counter
orchestrator	orchestrator_orphans_recovered_total	Counter
orchestrator	orchestrator_active_tests	Gauge
telemetry-ingester	telemetry_events_processed_total	Counter
telemetry-ingester	telemetry_events_correct_total	Counter
telemetry-ingester	telemetry_kafka_consumer_lag	Gauge
bot-fleet	bot_fleet_orders_sent_total	Counter (persona, type, contestant)
bot-fleet	bot_fleet_order_latency_milliseconds	Histogram
bot-fleet	bot_fleet_active_tests	Gauge
bot-fleet	bot_fleet_kafka_events_dropped_total	Counter
leaderboard-api	leaderboard_websocket_connections	Gauge

Table 5: Prometheus metric inventory

14.2 Structured

Logging

All services log structured JSON in production and structured text in development. Every request log record carries `service`, `request_id`, `contestant_id`, and `test_id` dimensions where applicable, enabling a single test run to be traced across every service.

Logging levels follow the `LOG_LEVEL` environment variable: `debug`, `info`, `warn`, `error`. 5xx responses log at `ERROR`, 4xx at `WARN`, 2xx at `INFO`.

14.3 Distributed

Tracing

An OpenTelemetry collector (`infra/k8s/tracing/otel-collector.yaml`) receives OTLP traces from all services and forwards them to a Jaeger backend (`infra/k8s/tracing/jaeger.yaml`). Traces are exported via the OTLP HTTP exporter on port 4317.

14.4 Grafana

Dashboards

Two dashboards are provided in `infra/grafana/dashboards/`:

Platform Overview Active tests, orders per second, telemetry events dropped per second, p99 latency across all contestants, active bots.

Live Competition Per-contestant p99 latency time series, orders per second per contestant, correctness rate per contestant as a bar gauge.

Dashboards refresh every 5 seconds with a 15-minute lookback window during contest events.

15 Additional

Features

15.1 Anomaly

Detection

The `anomaly` package provides two layers of anomaly detection:

Rule-based detector Flags p99 below $1\mu\text{s}$ with more than 100 orders (suggesting pre-computed responses) and perfect correctness with sub- $10\mu\text{s}$ p99 (suggesting caching or replay). Reports are published to Redis as a capped list of 100 entries per contestant.

Welford online z-score detector Maintains a running mean and variance per contestant using Welford's one-pass algorithm. After a 100-sample warmup, any latency sample more than 4 standard deviations from the contestant's own baseline is flagged as a latency outlier.

15.2 Score

Prediction

The `Predictor` in `scorer/prediction.go` keeps up to 60 score samples per contestant.

On each scoring cycle it fits a least-squares linear regression over the sample index, projects the score 10 cycles ahead, and reports the predicted final score with a confidence value proportional to the number of samples collected. The trend is classified as `up`, `down`, or `flat` based on the regression slope.

15.3 Contestant

Insights

The **GET /v1/contestants/{id}/insights** endpoint computes the current standing of the contestant relative to the active field, identifies the weakest dimension (correctness, throughput, or latency), and generates auto-populated improvement tips. The response includes the contestant's current rank, score, TPS, and p99 latency alongside field size.

15.4 Commentary

Ticker

The **Generator** in `commentary/generator.go` compares successive leaderboard snapshots and emits human-readable ticker events:

- New contestant enters the board.
- Contestant climbs more than 2 positions.
- Contestant drops any position.
- Contestant crosses 10,000 orders per second.
- Top 3 contestants are separated by fewer than 5 points.

Ticker events ride the same Redis pub/sub channel as leaderboard updates, distinguished by `"type": "ticker_event"`. The frontend renders them as a horizontal scrolling ticker. The last 50 events are kept in a Redis capped list.

15.5 Webhooks

Contestants can register up to 10 HMAC-SHA256-signed webhook callbacks for the events `build_complete`, `test_complete`, `score_update`, and `anomaly`. The **Manager** in `webhooks/manager.go` delivers payloads asynchronously with exponential-backoff retries (up to 3 attempts, squared-second delays). The HMAC signature is included in the `X-TradeEval-Signature` header.

15.6 Analysis

API

The telemetry ingester exposes two historical analysis endpoints (guarded by `X-Internal-Token`):

GET /v1/analysis/latency-distribution Returns histogram-bucketed latency counts for the last hour for a given contestant.

GET /v1/analysis/head-to-head Compares two contestants' p99 latency over the last hour and declares a winner.

16 Infrastructure

as

Code

16.1 Docker

Compose

(Development)

Two Compose files are provided:

docker-compose.yml Full stack with all services and infrastructure. All services share a common `x-dev-env` YAML anchor that sets defaults for every environment variable

with fallback to sensible local values. The orchestrator database DSN is hardcoded to the container-internal address to prevent a misconfiguration where the host-port DSN from `.env` would be used inside a container.

docker-compose.dev.yml Infrastructure only (Kafka, Redis, MinIO, TimescaleDB, PostgreSQL). Go services are run by hand with `go run .` for faster iteration cycles.

A one-shot **db-migrate** service applies all SQL migrations in lexical order to both databases on startup, then exits. A one-shot **kafka-init** service creates all Kafka topics if they do not already exist. Health checks ensure dependent services wait for their prerequisites.

16.2 Helm

Charts

One Helm chart per service lives in `infra/helm/`. Each chart provides:

- A Deployment with configurable replica count, image pull policy, and resource requests and limits.
- A Service exposing the relevant port.
- Liveness and readiness probes pointing at `/healthz` and `/readyz`.
- An optional `HorizontalPodAutoscaler` targeting CPU utilization.
- A `preStop` hook with a `sleep 15` to allow in-flight requests to complete during rolling updates.

The umbrella chart at `infra/helm/trade-eval-platform/` lists the orchestrator as a Helm dependency. The namespace manifest enforces `pod-security.kubernetes.io/enforce: baseline`.

16.3 Kubernetes

Manifests

Raw manifests in `infra/k8s/` cover:

Network policies A default-deny policy (`infra/k8s/security/deny-all-default.yaml`) blocking all ingress and egress. Per-service allow rules open only the ports and peers each service legitimately needs. The contestant network policy (`infra/k8s/security/network-policy.yaml`) allows ingress only from **bot-fleet** and **build-worker** on port 8080, with empty egress.

Monitoring ServiceMonitors for Prometheus operator scraping, PrometheusRule alert rules for high Kafka lag, dropping telemetry events, and stuck tests.

KEDA ScaledObject Scales the bot-fleet deployment to zero when the `orchestrator-events` topic has no lag, and up to 100 replicas proportional to lag.

RBAC Service accounts per service; the build-worker gets pod management permissions in the `trade-eval` namespace for container lifecycle.

16.4 Terraform

Cloud infrastructure is provisioned in `infra/terraform/` across:

- `vpc.tf` – VPC, subnets, NAT gateway, route tables.

- **eks.tf** – EKS cluster with two node groups: a system group (t3.medium) and a bot-fleet group (c6i.2xlarge) with a `dedicated=bot-fleet` taint.
- **databases.tf** – RDS PostgreSQL (orchestrator), RDS PostgreSQL with TimescaleDB (telemetry), ElastiCache Redis, MSK Kafka.
- **s3.tf** – Submissions bucket with public access block and 7-day lifecycle.
- **backend.tf** – S3 remote state with DynamoDB locking and AES-256 encryption.

Remote state is encrypted and locked. Workload identity (IAM Roles for Service Accounts) is used so no long-lived cloud credentials live in pod specifications. Secrets are sourced from a secrets manager, never committed to the repository.

17 CI/CD

Pipeline

Three GitHub Actions workflows in `.github/workflows/` guard the main branch.

17.1 Continuous Integration (ci.yml)

1. **Go vet**: runs `go vet ./...` across every service module in parallel via a matrix build.
2. **Go tests**: runs `go test ./...` for each service in a separate matrix job with Go 1.26. The `cache-dependency-path` is set per service for efficient cache hits.
3. **Frontend**: installs dependencies, type-checks with TypeScript, runs Vitest component tests, and builds the production bundle.
4. **Build and push** (main branch only): builds Docker images for all 6 Go services and the frontend, and pushes them to GHCR tagged with the short commit SHA.

All CI jobs use `concurrency` groups with `cancel-in-progress: true` to cancel redundant runs on rapid pushes.

17.2 Integration (integration.yml)

Triggered on pushes to main and on `workflow_dispatch`. Steps:

1. Free all host ports used by the stack.
2. Start the full Compose stack with `-build`.
3. Wait up to 180 seconds for `GET /v1/health` to return 200.
4. Run `scripts/integration-test.sh`: uploads the reference C++ order book, waits for `ready`, starts a 30-second test with 10 bots, waits for `completed`, and verifies the leaderboard has at least one entry.
5. Tear down the stack.

17.3 Security

(`security.yml`)

Runs on pushes to main and on a weekly Monday schedule:

1. **Trivy**: builds the submission-api image and scans it for HIGH and CRITICAL CVEs.
2. **gosec**: runs static security analysis across all Go modules.
3. **Dependency audit**: runs `go mod verify` and `npm audit --audit-level=high`.

18 Chaos

Engineering

The platform is tested against the failures it is built to survive. Three chaos scenarios are validated:

Contestant server failure The `ControllableMockServer` in `chaos/mock_server.go` can inject configurable delays and error rates. Tests in `chaos/chaos_test.go` verify that the circuit breaker opens after sustained failures, the bot fleet stays bounded, and latency measurement degrades gracefully rather than hanging indefinitely.

Pod restart during test The orchestrator's heartbeat and orphan detection are exercised by the shell script `scripts/chaos/chaos-experiments.sh`. After restarting the orchestrator container, the script waits `ORPHAN_DETECTION_SECONDS` and verifies that previously-running tests transition to completed rather than remaining stuck in running.

Telemetry backpressure The ingester's bounded buffer (50,000 events) and the bot-fleet producer's channel (10,000 events) are sized to absorb burst. When the ingester falls behind, the bot-fleet producer increments the `bot_fleet_kafka_events_dropped_total` counter rather than blocking bot goroutines. The Kafka lag monitor alerts when consumer lag exceeds 100,000 events.

19 Architecture

Decision

Records

ADR-1: Correctness is decided by a platform-owned reference, never by the submission

Decision: The shadow order book in the telemetry ingester is the sole authority on what the correct fill should have been.

Rationale: A contestant controls the code and is motivated to return fabricated fills. Trusting self-reported fills would make the correctness score trivially gameable.

Trade-off: The platform must maintain a correct, bug-free matching engine. Any bug in the reference engine is a scoring bug.

ADR-2: Correctness and concurrency are measured separately

Decision: A serialized correctness phase (one in-flight request) produces the correctness rate. A concurrent load phase produces throughput and latency. The two

phases are separated in time by sequence number.

Rationale: The true processing order of concurrent requests is not observable from outside. Judging correctness under concurrency requires tolerating ordering-sensitive disagreements, which creates a loophole. Judging in a serialized phase eliminates the ordering ambiguity entirely.

Trade-off: A slightly more complex two-phase test plan. Benefit: the correctness rate is not a permissive guess.

ADR-3: Kafka offsets commit after the handler, not on read

Decision: The telemetry consumer fetches messages, enqueues them in the reorder buffer, and commits offsets only after the buffer flushes and the pipeline writes the events to TimescaleDB.

Rationale: Auto-commit marks events consumed before they are persisted. A crash after auto-commit but before the write silently drops the event and corrupts scores.

Trade-off: At-least-once delivery requires query-time deduplication. Benefit: no silent event loss on restart.

ADR-4: Percentiles use a bounded logarithmic histogram, not sorting

Decision: Latency percentiles are computed from an HDR histogram (18 KB per contestant, $O(1)$ record, single scan for a percentile).

Rationale: Sorting is $O(n \log n)$ per window and requires keeping every sample in memory. At 1 million events per second across many contestants, sorting eventually exhausts heap and falls behind Kafka.

Trade-off: Percentiles are approximate to a fixed relative precision (about 1%). Benefit: constant-time recording, bounded memory, no head-of-line blocking in the pipeline.

ADR-5: The contestant container has no network path to internal infrastructure

Decision: Contestant containers are attached only to the `contestant-isolated` network. The platform network is never assigned to contestant containers.

Rationale: Network isolation is the defense that requires no cooperation from the contestant's code. Even if every other sandboxing layer is bypassed, there is no path to Kafka, Redis, PostgreSQL, or MinIO.

Trade-off: Build-worker and bot-fleet must be dual-homed. Benefit: a sandbox escape reaches nothing of value.

ADR-6: Sandbox controls fail closed

Decision: If a required security profile (seccomp, AppArmor) cannot be read, or in strict mode is not configured, the launcher returns an error and the build fails.

Rationale: Silently running hostile code without its confinement profiles is worse than a failed build. Operators should know immediately if a profile is missing.

Trade-off: A misconfigured profile blocks builds until fixed. Benefit: the platform never silently runs unconfined contestant code.

ADR-7: The WebSocket fan-out drops slow clients rather than blocking

Decision: The hub performs a non-blocking send; if a client's buffer is full, the hub closes and drops that client.

Rationale: One slow viewer (for example, on a congested mobile connection) must not stall the broadcast for all other viewers.

Trade-off: A slow viewer misses frames and reconnects. Benefit: one slow client cannot stall the broadcast for thousands of others.

ADR-8: The leaderboard read path scales through Redis pub/sub

Decision: The scorer publishes to `leaderboard:updates`; every pod subscribes independently.

Rationale: Direct broadcasting from the scorer to its own clients, combined with forwarding to other pods, creates double-delivery and tight coupling between scorer and pod count.

Trade-off: A frame can be dropped at the pub/sub layer under burst. Benefit: pods are stateless and horizontally scalable, and the next tick corrects any dropped frame.

ADR-9: The orchestrator recovers tests through heartbeats and orphan detection

Decision: Each orchestrator instance writes a heartbeat timestamp to the test row. A second instance detects stale heartbeats and re-registers the stop timer or fails the test.

Rationale: Kubernetes restarts pods without warning. Without recovery, a pod crash during a test would leave the test stuck in `running` forever.

Trade-off: Recovery takes up to `ORPHAN_DETECTION_SECONDS` (default 60 seconds). Benefit: a pod restart is a brief blip, not a permanently stuck test.

20 Technology

Decisions

Go for all services Predictable latency without a stop-the-world garbage pause long enough to distort measurement, cheap goroutine concurrency for the bot fleet and fan-out hub, and static binaries that fit minimal containers. All modules are pinned to Go 1.26.0.

React and TypeScript for the frontend A live, diff-animated leaderboard with a re-connecting socket is straightforward in React. The type system catches contract drift against service schemas at build time. Recharts provides the latency charts and SVG correctness gauge.

Kafka for the durable backbone Replayable, partitioned, at-least-once delivery is the property that makes scoring trustworthy through restarts. Redpanda is a drop-in alternative for lighter operational footprint.

Redis for the live path Per-second metric writes and pub/sub fan-out need in-memory speed. Redis provides a native publish-subscribe primitive alongside the hash, set,

and string operations the platform uses.

TimescaleDB for history The latency sample table is the highest-volume table in the platform. A time-partitioned hypertable with columnar compression and auto-chunking is the right tool, and it keeps all analysis queries in SQL.

A logarithmic-bucket histogram for percentiles Constant-time recording and bounded memory are the only way the percentile path keeps up at peak event rates with many concurrent contestants. The 18 KB per-contestant footprint is the property that lets the ingester scale to the full field.

pgx and COPY for TimescaleDB ingestion The `pgx/v5` library's `CopyFrom` sustains row rates that batched INSERT statements cannot match. At peak ingest rates during a contest, this is the difference between the ingester keeping up and falling behind.

chi router for submission-api The `go-chi/chi` router provides lightweight, standard-library-compatible routing with built-in middleware for request IDs, logging, recovery, and timeouts.

21 Performance

Characteristics

Design targets shaped by contest requirements:

Ingestion Must keep Kafka consumer lag bounded under the full bot-fleet event rate. The bottleneck is the percentile and validation path; both are constant-time per event. The consumer group can be scaled out across the 16 `bot-telemetry` partitions by adding replicas.

Memory per contestant (ingester) Bounded at approximately 18 KB for the HDR histogram buckets plus counters. Memory footprint grows linearly with field size and is dominated by the histogram, not by event volume.

Build time Expected under 60 seconds for a typical C++ submission on a warm runner. The 600-second build timeout accounts for cold base-image pulls.

Leaderboard freshness One scoring interval (default 500 ms) from event to browser display. The WebSocket fan-out design targets many thousands of concurrent viewers per pod.

API latency (submission-api) The p99 request duration target is under 2 seconds, verified by the k6 load test (`scripts/load-test/submission_api_load.js`).

22 Security

Model

Summary

22.1 Primary

Adversary:

Contestant

The contestant controls arbitrary code inside a sandbox and may attempt to:

- Escape the container via kernel exploits (mitigated by `seccomp`, `AppArmor`, dropped capabilities, `no-new-privileges`).

- Reach internal infrastructure once escaped (mitigated by network isolation: no route from the contestant network to the platform network).
- Cheat the scoring by fabricating fills (mitigated by the reference order book and the serialized correctness phase).
- Exhaust shared resources (mitigated by memory, CPU, and process count caps).

22.2 Secondary Adversary: External Attacker

The submission API and leaderboard API face the internet in production. Mitigations:

- API key authentication on all mutating endpoints.
- Cross-contestant reads return 404 (not 403), so existence is not leaked.
- Per-contestant submission rate limit and per-IP leaderboard rate limit, both enforced in Redis.
- Zip-slip guard and decompression-bomb detection on uploads, applied both at the header level and at the byte level during extraction.
- Constant-time comparison for all API keys and admin tokens via `crypto/subtle.ConstantTimeCompare`.
- Security headers (`X-Content-Type-Options`, `X-Frame-Options`, `Content-Security-Policy`, etc.) applied by middleware on every response.

23 Contest Day Runbook

23.1 24 Hours Before

- Verify `make test` is green across all modules and the frontend.
- Verify `make build` succeeds for every image.
- Validate `docker compose config` produces no errors.
- Seed contestants (`make seed`) and distribute API keys.
- Confirm Kafka topic partition counts (`infra/kafka/topics.sh`).
- Run smoke test on staging (`make smoke-test`).

23.2 One Hour Before

- Confirm all pods running: `kubectl get pods -n trade-eval`.
 - Confirm Kafka consumer lag near zero: `make check-kafka-lag`.
 - Confirm TimescaleDB and Redis are reachable.
 - Scale the bot fleet to zero: `kubectl get hpa bot-fleet -n trade-eval`.
-

23.3 During the Contest

Every 15 minutes:

- Kafka consumer lag under 1,000 events.
- No OOMKilled events in the contestant namespace.
- `leaderboard:updated_at` is fresh.
- No errors in submission-api or orchestrator logs.

23.4 Incident Response

Stale leaderboard Check leaderboard-api logs and the `leaderboard:cached` key in Redis, then `kubectl rollout restart deployment/leaderboard-api`.

Build stuck Check build-worker logs and the Docker daemon; builds are bounded to 600 seconds.

Growing Kafka lag `kubectl scale deployment telemetry-ingester --replicas=6`.

Orchestrator crash Kubernetes restarts it. Orphan detection recovers running tests within approximately 60 seconds. Verify via `/admin/active-tests`.

23.5 Close

1. Wait for all active tests to complete.
2. Freeze the leaderboard: `POST /admin/v1/leaderboard/freeze` with the admin key.
3. Export results and announce winners.
4. Scale the bot fleet to zero: `kubectl scale deployment bot-fleet --replicas=0`.

24 Correctness Verification

All six Go service modules and the frontend are fully verified:

Module	Result	Notable Tests
telemetry-ingester	PASS	shadowbook (9), latency HDR + precision (8), metrics aggregator (4), consumer commit semantics (3), reorder buffer (3), storage flush (3)
bot-fleet	PASS	shadow book (5), FIX protocol (4), benchmarks (3), chaos (3)
leaderboard-api	PASS	scorer (4), hub (5)
orchestrator	PASS	scorer normalization (3)
submission-api	PASS	handlers (3), hash pinning (2)
build-worker	PASS	build strategies (4)
frontend	PASS	LeaderboardTable (3), type-check, build

Module	Result	Notable Tests
--------	--------	---------------

Table 6: Test verification summary

24.1 Key	Test	Coverage	Areas
HDR Histogram Precision	The <code>TestPercentileAccuracy</code> test records a uniform ramp from 1 to 100,000 and asserts that p50, p90, p99, and p99.9 all land within 2% relative error of the true values. <code>TestMemoryBounded</code> asserts the counts array stays below 64 KB regardless of event volume.		
Serialized Correctness Script	<code>TestSerializedCorrectRunScoresOne</code> runs the full 13-step correctness script through the validator with correct actual fills and asserts all results are <code>Correct: true</code> .		
Cheater Detection	<code>TestAlwaysFilledCheaterScoresNearZero</code> runs the same script with FILLED returned for every order and asserts that at most 2 orders (the genuinely FILLED ones) score correctly.		
Hard Violations	Individual tests verify each hard violation type: overfill, impossible price, impossible cross, fill with no liquidity, and negative quantity.		
Consumer Commit Semantics	<code>TestConsumerCommitsOnlyAfterFlush</code> and <code>TestConsumerCrashLe</code> use a fake Kafka source to assert the commit-after-persist guarantee.		
WebSocket Hub	Five hub tests cover: slow client drop, max client enforcement, new client receiving the last broadcast, broadcast delivery to all clients, and clean unregistration.		
TPS Counter	Tests verify counting, stale-after-idle zero return, and wrap-around non-collision.		
API Key Hashing	<code>TestSeededHashesMatch</code> pins the SHA-256 hashes of all 5 seed API keys to the values in the seed SQL, ensuring the lookup path stays consistent with the seed data.		

25 Design

Principles

The following principles govern the platform's architecture and scoring model and inform every component decision.

Single source of truth for correctness The reference engine and the published API contract are kept in lockstep. The contract explicitly pins every edge case (sweep pricing, cancel-of-filled status), so the reference and the contract always agree. Contestants test locally against the same reference the platform uses.

Serialized correctness, concurrent performance The platform measures correctness in a serialized phase where the true processing order is known and the comparison is exact. Throughput and latency are measured under full concurrency. This separation produces the most accurate results for both dimensions.

Fair, consistent latency measurement Latency is the HTTP round trip from the bot to the sandbox. Every contestant runs under the same conditions on the same infrastructure, so the shared network component cancels out in the ranking. This produces a fair comparative measurement.

Relative scoring for a live leaderboard Min-max normalization scores each contestant relative to the current field, which is the correct model for a live competitive leaderboard. The final ranking is captured at the freeze point.

Two deployment targets The Compose stack provides a fully functional development environment with all security controls except AppArmor. The Kubernetes deployment provides production-grade isolation, autoscaling, network policies, and pod security profiles. Both targets are provisioned as code and tested in CI.

Infrastructure as code Every sandboxing profile, network policy, Helm chart, Terraform resource, and SQL migration is version-controlled alongside the services they govern. A green CI run reproduces a known-good state.

Fail closed Security controls (seccomp, AppArmor, image scanning) fail closed: a missing or misconfigured control blocks the build rather than allowing an unprotected sandbox to start.

A Environment Variable Reference

Variable	Default	Description
KAFKA_BROKERS	localhost:9092	Comma-separated broker addresses
BUILD_JOBS_TOPIC	build-jobs	Build job Kafka topic
BOT_TELEMETRY_TOPIC	bot-telemetry	Telemetry Kafka topic
ORCHESTRATOR_EVENTS_TOPIC	orchestrator-events	Orchestrator lifecycle topic
REDIS_URL	localhost:6379	Redis connection address
MINIO_ENDPOINT	localhost:9000	MinIO server endpoint
MINIO_ACCESS_KEY	minioadmin	MinIO access key
MINIO_SECRET_KEY	minioadmin	MinIO secret key
MINIO_BUCKET	submissions	Submission artifacts bucket
TIMESCALE_DSN	postgres://...	DSN for TimescaleDB
ORCHESTRATOR_DB_DSN	postgres://...	DSN for orchestrator PostgreSQL
SUBMISSION_API_PORT	8080	HTTP port for submission-api
MAX_UPLOAD_SIZE_MB	50	Maximum upload size in MB
MAX_CONCURRENT_BUILDS	3	Parallel container builds per worker
BUILD_TIMEOUT_SECONDS	120	Build timeout before kill
WORK_DIR	/tmp/builds	Local temporary build directory
HEARTBEAT_INTERVAL_SECONDS	10	Orchestrator heartbeat frequency

Variable	Default	Description
ORPHAN_DETECTION_SECONDS	60	Stale heartbeat threshold
AUTO_TRIGGER_TESTS	false	Auto-start tests on build ready
MAX_BOTS_PER_POD	200	Maximum bots per bot-fleet pod
BOT_REQUEST_TIMEOUT_MS	5000	Per-bot request timeout
TEST_DEFAULT_DURATION_SECONDS	30	Default test duration
TEST_DEFAULT_BOT_COUNT	500	Default bot count per test
WINDOW_SIZE_SECONDS	30	Tumbling window for aggregation
TELEMETRY_BATCH_SIZE	500	Events to batch before TimescaleDB flush
FLUSH_INTERVAL_MS	1000	Maximum time between flushes
REORDER_BUFFER_MS	100	Reorder window duration
LEADERBOARD_API_PORT	8084	HTTP/WebSocket port for leaderboard-api
LEADERBOARD_UPDATE_INTERVAL_MS	500	Scoring and broadcast interval
MAX_WEBSOCKET_CONNECTIONS	10000	Maximum concurrent WebSocket clients
SANDBOX_MEMORY_LIMIT	512m	Docker memory limit for sandboxes
SANDBOX_CPU_CORES	2,3	CPU cores pinned to sandboxes
CONTESTANT_NETWORK	contestant-isolated	Docker network for contestant containers
SECCOMP_PROFILE	(empty)	Path to seccomp JSON profile
APPARMOR_PROFILE	(empty)	Name of AppArmor profile
SANDBOX_STRICT	false	Require all controls in production
ADMIN_API_KEY	(change this)	Admin endpoint API key
INTERNAL_API_TOKEN	internal-dev-token	Internal service token
ENVIRONMENT	development	Controls log format
LOG_LEVEL	info	Minimum log level

Table 7: Complete environment variable reference

B Make Targets Reference

Target	Description
<code>make up</code>	Start the full local stack
<code>make dev</code>	Start infrastructure only (run services with <code>go run</code>)
<code>make down</code>	Stop all containers
<code>make restart</code>	Restart all containers
<code>make logs SVC=...</code>	Tail logs for a specific service

Target	Description
<code>make migrate</code>	Apply SQL migrations to both databases
<code>make seed</code>	Seed local contestants
<code>make build</code>	Build Docker images for all services and frontend
<code>make test</code>	Run unit tests for every Go module
<code>make test-race</code>	Run tests with the race detector
<code>make test-integration</code>	Run integration tests (requires Docker)
<code>make bench</code>	Run benchmarks
<code>make lint</code>	Run <code>go vet</code> and <code>golangci-lint</code>
<code>make proto</code>	Regenerate protobuf Go code
<code>make template</code>	Generate contestant starter code
<code>LANG=X</code>	
<code>make smoke-test</code>	End-to-end check against a running stack
<code>make verify</code>	Full platform verification
<code>make security-scan</code>	Scan images and Go code
<code>make clean</code>	Tear down stack and remove coverage files
<code>make help</code>	Show all available targets

Table 8: Make target reference

C Scorer

Normalization

Example

Given three contestants with the following raw metrics:

Contestant	TPS	p99 (μ s)	Correctness
Alice	9,000	450	0.999
Bob	5,000	1,200	0.999
Carol	11,000	620	0.995

Normalization:

$$\text{norm_TPS}(\text{Alice}) = \frac{9000 - 5000}{11000 - 5000} = 0.667 \quad (2)$$

$$\text{norm_TPS}(\text{Bob}) = \frac{5000 - 5000}{11000 - 5000} = 0.000 \quad (3)$$

$$\text{norm_TPS}(\text{Carol}) = \frac{11000 - 5000}{11000 - 5000} = 1.000 \quad (4)$$

$$\text{norm_p99}(\text{Alice}) = \frac{450 - 450}{1200 - 450} = 0.000 \quad (5)$$

$$\text{norm_p99}(\text{Bob}) = \frac{1200 - 450}{1200 - 450} = 1.000 \quad (6)$$

$$\text{norm_p99}(\text{Carol}) = \frac{620 - 450}{1200 - 450} = 0.227 \quad (7)$$

Composite scores (Equation 1):

$$\text{score}(\text{Alice}) = (0.40 \times 0.667 + 0.40 \times (1 - 0.000) + 0.20 \times 0.999) \times 100 = 86.7 \quad (8)$$

$$\text{score}(\text{Bob}) = (0.40 \times 0.000 + 0.40 \times (1 - 1.000) + 0.20 \times 0.999) \times 100 = 20.0 \quad (9)$$

$$\text{score}(\text{Carol}) = (0.40 \times 1.000 + 0.40 \times (1 - 0.227) + 0.20 \times 0.995) \times 100 = 84.8 \quad (10)$$

Final ranking: Alice (86.7), Carol (84.8), Bob (20.0).